数据结构：不相交集 ADT
复习讲义

目录

1 等价关系 (Equivalence Relations) 2
1.1 关系 . . . 2
1.2 等价关系 . . . 2
1.3 等价类 . . . 2

2 动态等价问题 (Dynamic Equivalence Problem) 2
2.1 问题 . . . 2
2.2 Union / Find 算法框架 . . . 2

3 基本数据结构 (Basic Data Structure) 3
3.1 集合与森林表示 . . . 3
3.2 操作 . . . 3
3.3 普通 Union . . . 3
3.4 数组实现 . . . 3
3.5 普通算法的坏情况 . . . 4

4 聪明的合并算法 (Smart Union Algorithms) 4
4.1 按大小合并 (Union-by-Size) . . . 4
4.2 按高度合并 (Union-by-Height) . . . 4

5 路径压缩 (Path Compression) 5
5.1 递归版 Find . . . 5
5.2 迭代版 Find . . . 5
5.3 路径压缩的作用 . . . 5

6 最坏情况复杂度 5
6.1 Tarjan 引理 . . . 5
6.2 $\log^* N$ 与反 Ackermann 函数 . . . 6

7 全章总结 6

1 等价关系 (Equivalence Relations)

1.1 关系

在集合 S 上定义关系 R：对任意元素对 (a, b)，其中 $a, b \in S$，命题 aRb 要么为真，要么为假。若 aRb 为真，则称 a 与 b 相关。

1.2 等价关系

集合 S 上的关系 $\sim$ 是等价关系，当且仅当它满足：

1. **自反性 (reflexive)**：对任意 $a \in S$，有 $a \sim a$。
2. **对称性 (symmetric)**：若 $a \sim b$，则 $b \sim a$。
3. **传递性 (transitive)**：若 $a \sim b$ 且 $b \sim c$，则 $a \sim c$。

1.3 等价类

集合 S 中两个成员 x, y 属于同一个等价类，当且仅当 $x \sim y$。等价关系会把集合划分成若干个互不相交的等价类。

2 动态等价问题 (Dynamic Equivalence Problem)

2.1 问题

给定等价关系 $\sim$，需要动态回答任意 a, b 是否满足 $a \sim b$。

课件例子：

$$S = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12\}$$

给定 9 个关系：

$$12 \equiv 4,\ 3 \equiv 1,\ 6 \equiv 10,\ 8 \equiv 9,\ 7 \equiv 4,\ 6 \equiv 8,\ 3 \equiv 5,\ 2 \equiv 11,\ 11 \equiv 12.$$

最终等价类为：

$$\{2, 4, 7, 11, 12\}, \quad \{1, 3, 5\}, \quad \{6, 8, 9, 10\}.$$

2.2 Union / Find 算法框架

```
Initialize N disjoint sets;
while ( read in a ~ b ) {
    if ( Find(a) != Find(b) )
        Union the two sets;
}

while ( read in a and b ) {
```

```
    if ( Find(a) == Find(b) )
        output true;
    else
        output false;
}
```

该问题是动态的 (on-line)：关系逐条读入并合并，查询也可随时到来。

3 基本数据结构 (Basic Data Structure)

3.1 集合与森林表示

元素通常编号为 $1, 2, \ldots, N$。集合 $S_1, S_2, \ldots$ 两两不相交，即：

$$S_i \cap S_j = \varnothing, \quad i \neq j.$$

课件例子：

$$S_1 = \{6, 7, 8, 10\}, \quad S_2 = \{1, 4, 9\}, \quad S_3 = \{2, 3, 5\}.$$

可以用森林表示集合：每棵树代表一个集合，指针从孩子指向父结点，根代表集合名。

3.2 操作

- **Union(i,j)**：将 S_i 与 S_j 替换为并集 $S = S_i \cup S_j$。
- **Find(i)**：找到包含元素 i 的集合 S_k，通常返回该集合的根。

3.3 普通 Union

Union 的基本思想：把一棵树的根指向另一棵树的根。

```
void SetUnion( DisjSet S, SetType Rt1, SetType Rt2 )
{
    S[ Rt2 ] = Rt1;
}
```

这里把以 `Rt2` 为根的树接到 `Rt1` 下。

3.4 数组实现

课件的数组实现：

- `S[element]` 存放该元素的父结点。
- 对根结点，初始简单实现中可令 `S[root] = 0`，集合名为根下标。
- 因元素编号为 1 到 N，可直接作为数组下标。

普通 Find：

```
SetType Find( ElementType X, DisjSet S )
{
    for ( ; S[X] > 0; X = S[X] )
        ;
    return X;
}
```

3.5 普通算法的坏情况

Union 和 Find 总是成对出现，因此要分析一串操作。若反复执行：

```
union(2, 1), find(1);
union(3, 2), find(1);
...
union(N, N-1), find(1);
```

可能构造出高度为 $N-1$ 的链，每次 `find(1)` 都很慢，整体可达：

$$T = \Theta(N^2).$$

4 聪明的合并算法 (Smart Union Algorithms)

4.1 按大小合并 (Union-by-Size)

规则：总是把较小的树接到较大的树下。课件用：

```
S[Root] = -size
```

记录以根为代表的集合大小，初始为 -1。

Union-by-Size 引理

由 union-by-size 创建的含 N 个结点的树，其高度至多为 $O(\log N)$。
证明思想：某个元素每次深度增加时，它所在集合至少翻倍，因此最多增加 $\log_2 N$ 次。

因此，N 次 Union 和 M 次 Find 的时间复杂度为：

$$O(N + M \log_2 N).$$

4.2 按高度合并 (Union-by-Height)

规则：总是把较浅的树接到较深的树下。若两棵树高度相等，合并后高度加 1。课件要求阅读教材 Figure 8.13 的详细实现。

实际实现中常把高度称为 rank。rank 是控制树高的估计值，不一定总等于真实高度，尤其在路径压缩之后。

5 路径压缩 (Path Compression)

5.1 递归版 Find

```
SetType Find( ElementType X, DisjSet S )
{
    if ( S[X] <= 0 )
        return X;
    else
        return S[X] = Find( S[X], S );
}
```

路径压缩在查找根的同时，把路径上的每个结点直接接到根上。

5.2 迭代版 Find

```
SetType Find( ElementType X, DisjSet S )
{
    ElementType root, trail, lead;
    for ( root = X; S[root] > 0; root = S[root] )
        ;
    for ( trail = X; trail != root; trail = lead ) {
        lead = S[trail];
        S[trail] = root;
    }
    return root;
}
```

5.3 路径压缩的作用

单次 Find 可能因为额外修改指针略慢，但对一串操作会显著加速。课件提醒：路径压缩会改变真实高度，因此不能与“精确高度”直接兼容；实际使用 union-by-rank，把 rank 当作高度估计。

6 最坏情况复杂度

6.1 Tarjan 引理

令 $T(M,N)$ 是处理一串混合操作的最大时间，其中包含 $M \geq N$ 次 Find 和 $N-1$ 次 Union。使用 union-by-rank 加路径压缩时，存在正常数 k_1, k_2，使得：

$$k_1 M\alpha(M,N) \leq T(M,N) \leq k_2 M\alpha(M,N).$$

这里 $\alpha(M,N)$ 是 Ackermann 函数的某种反函数，增长极慢。

6.2 log-star 与反 Ackermann 函数

课件给出直观比较：$\log^* N$ 表示对 N 连续取对数直到结果不超过 1 所需次数。例如：

$$\log^* 2^{65536} = 5.$$

在所有现实规模下，$\alpha(M, N)$ 都非常小，通常可近似看成常数。因此并查集在实践中几乎是均摊常数时间。

7 全章总结

1. 等价关系满足自反、对称、传递，会把集合划分为等价类。
2. 动态等价问题用 Union / Find 在线维护等价类。
3. 并查集通常用森林表示，每个集合由根代表。
4. 朴素合并可能形成长链，使一串操作达到 $\Theta(N^2)$。
5. union-by-size 或 union-by-height 可把树高控制在 $O(\log N)$。
6. 路径压缩会把查找路径直接接到根上，极大改善后续操作。
7. union-by-rank + path compression 的均摊复杂度为 $O(\alpha(M, N))$，实践中几乎为常数。

“并查集不关心集合内部顺序，只关心谁和谁属于同一个集合。”